

DOCUMENTATION — Pharmacie Backend (RDC)

Document de référence exhaustif. Il suffit de lire ce fichier pour comprendre l'intégralité du projet sans toucher au code source.

1. Vue d'ensemble du projet

Contexte métier

Ce backend alimente une application de **gestion de pharmacie digitale** adaptée au contexte de la **République Démocratique du Congo (RDC)**. Les pharmacies congolaises font face à plusieurs défis spécifiques :

- **Double devise** : les transactions se font aussi bien en **Franc Congolais (CDF)** qu'en **Dollars Américains (USD)**, parfois dans la même journée.
- **Traçabilité des ordonnances** : la falsification d'ordonnances est fréquente ; un système de validation numérique réduit ce risque.
- **Gestion du stock** : les ruptures de médicaments sont courantes ; un seuil d'alerte minimum évite les pénuries.
- **Personnel diversifié** : une pharmacie emploie des rôles différents (pharmacien, préparateur, caissier) avec des permissions distinctes.
- **Tickets de caisse numériques** : remplacer les reçus papier par des PDF avec QR code facilite les contrôles et remboursements.

Problèmes résolus par ce backend

Problème terrain	Solution technique
Gestion multi-rôles (Admin, Pharmacien, Préparateur, Caissier)	RBAC via <code>RolesGuard</code> + décorateur <code>@Roles()</code>
Sécurité des sessions (vol de token)	JWT en cookie HTTP-only (inaccessible en JavaScript)
Transactions financières en CDF et USD	Champ <code>devise</code> avec enum <code>Devise</code> sur chaque vente/ligne
Risque de survente (stock insuffisant)	Transaction interactive vérifiant le stock avant décrément
Traçabilité : ordonnance ↔ vente	Lien <code>ordonnanceId</code> unique sur <code>Vente</code> , statut <code>VALIDEE</code>
Tickets physiques falsifiables	PDF généré serveur-side + QR code encodant l'ID de vente
Fiabilité des réponses API	Format uniforme <code>{ success, data, message }</code> via Interceptor
Rupture de stock non détectée à temps	WebSocket Socket.IO — alerte <code>stock-alerte</code> émise en temps réel dès que le <code>stock ≤ seuil</code>
Création de compte staff non sécurisée	L'admin seul crée les comptes via <code>POST /users</code> — mot de passe temporaire généré et envoyé par email, changeable via <code>PATCH /auth/change-password</code>
Auto-attribution de rôle staff	<code>POST /auth/register</code> est exclusivement CLIENT — le champ <code>role</code> n'existe pas dans ce DTO

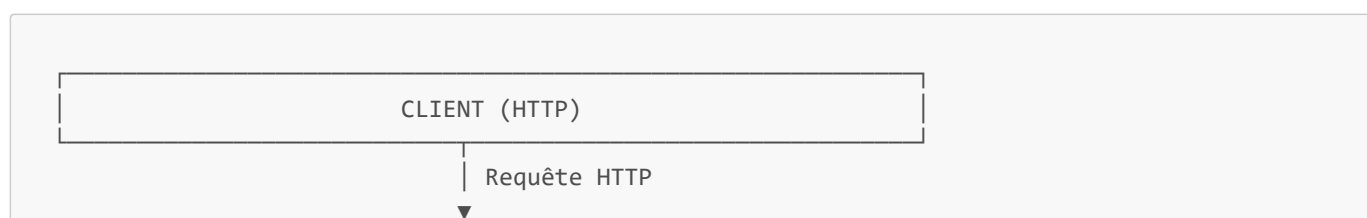
Problème terrain	Solution technique
Chicken-and-egg : premier admin	<code>POST /users</code> exige un ADMIN déjà existant, <code>POST /auth/register</code> force CLIENT — le premier admin est injecté via <code>npx prisma db seed</code> (script <code>prisma/seed.ts</code> idempotent)
Clients sans catalogue accessible	<code>GET /medicaments</code> et <code>GET /medicaments/:id</code> sont décorés <code>@Public()</code> — accessibles sans token pour que les clients puissent parcourir le catalogue
Réapprovisionnement sans traçabilité	<code>GET /stock/rapport-commande</code> génère une prévisualisation des produits sous seuil ; <code>POST /stock/rapport-commande/envoyer</code> génère un PDF ou Excel et l'envoi directement au fournisseur par email
Clients sans moyen de commander	<code>CommandesModule</code> expose <code>POST /commandes</code> (CLIENT) + routes de validation pour le staff

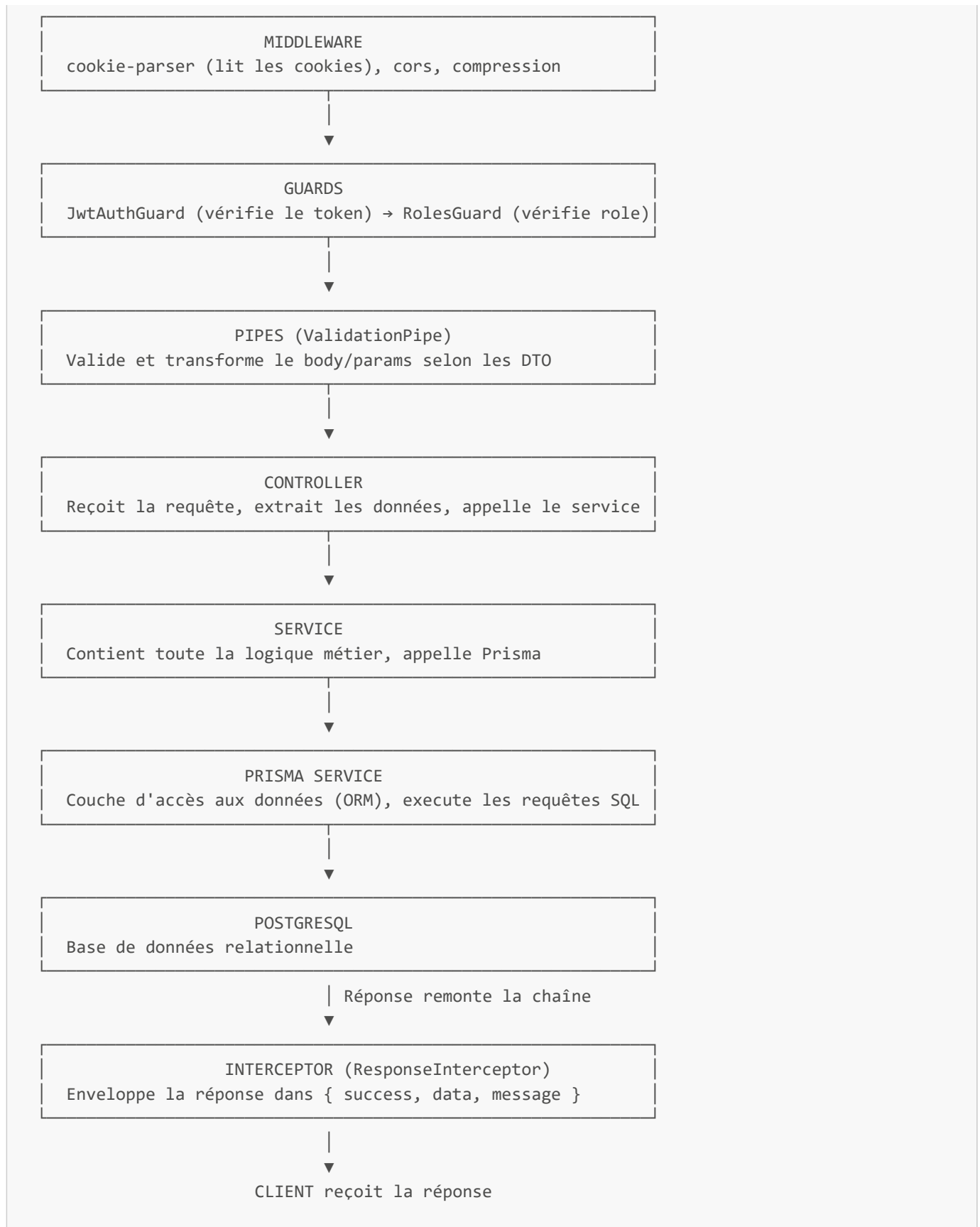
Choix techniques et justifications

Technologie	Pourquoi ce choix
NestJS	Framework Node.js opinionné, basé sur les décorateurs TypeScript, inspiré d'Angular. Impose une structure modulaire claire, idéale pour des équipes.
Prisma	ORM nouvelle génération avec un schéma déclaratif, une génération automatique de types TypeScript et des transactions fiables. Remplace Sequelize ou TypeORM.
PostgreSQL	Base de données relationnelle robuste, gère les types <code>Decimal</code> pour les montants financiers (pas de flottants), les enums, et les relations complexes.
JWT en cookie HTTP-only	Le token n'est pas accessible par JavaScript (protection XSS). Transmis automatiquement par le navigateur (pas besoin d'en-tête <code>Authorization</code>).
Passport.js	Bibliothèque d'authentification de référence pour Node.js. Gère les stratégies JWT de façon standardisée.
class-validator	Validation déclarative via décorateurs sur les DTO. Intégration native avec NestJS via <code>ValidationPipe</code> .
bcrypt	Hachage de mots de passe résistant aux attaques par force brute (facteur de coût configurable).
qrcode + pdfkit	Génération serveur-side de QR codes et PDF pour les tickets de caisse.
Swagger (@nestjs/swagger)	Documentation API automatique et interface de test intégrée, générée depuis les décorateurs du code.
Socket.IO (@nestjs/websockets)	WebSocket temps réel. Le serveur pousse les alertes de stock aux clients connectés sans polling. Fonctionne sur le même port que l'API REST.

2. Architecture générale

Schéma en couches





Organisation en modules NestJS

NestJS organise le code en **modules** autonomes et réutilisables. Chaque module encapsule ses controllers, services, et providers. Les modules peuvent s'importer mutuellement pour partager leurs providers.

```
AppModule (racine)
├── PrismaModule      → Partage PrismaService avec tous les modules
```

— AuthModule	→ Authentification, JWT, cookies
— UsersModule	→ Gestion du personnel
— PatientsModule	→ Gestion des patients
— MedicamentsModule	→ Catalogue des médicaments (GET public sans token)
— StockModule	→ Gestion des stocks (dépend de NotificationsModule)
— OrdonnancesModule	→ Gestion des prescriptions
— VentesModule	→ Processus de vente (dépend de QrCodeModule + NotificationsModule)
— QrCodeModule	→ Génération QR code + PDF
— MailModule	→ Envoi d'emails via Gmail SMTP (dépend de UsersModule)
— FournisseursModule	→ Gestion des fournisseurs
— CommandesModule	→ Commandes clients – création et validation
— NotificationsModule	→ WebSocket Socket.IO – alertes stock temps réel

Cycle de vie d'une requête HTTP dans NestJS

Exemple : **POST** /ventes (créer une vente)

1. Requête reçue par Express (sous NestJS)
2. Middleware cookie-parser lit les cookies et les met dans req.cookies
3. JwtAuthGuard lit le cookie access_token, vérifie la signature JWT
 - Si invalide → 401 Unauthorized (HttpExceptionFilter formate l'erreur)
 - Si valide → req.user = { sub: userId, role: 'CAISSIER', ... }
4. RolesGuard vérifie que req.user.role est dans @Roles('CAISSIER', 'ADMIN', ...)
 - Si non autorisé → 403 Forbidden
5. ValidationPipe reçoit le body, instancie CreateVenteDto
 - Valide chaque champ selon les décorateurs class-validator
 - Si invalide → 400 Bad Request avec détail des erreurs
6. VentesController.create(dto, req.user) est appelé
7. VentesService.create(dto, userId) est appelé
 - Transaction interactive Prisma démarre
 - Pour chaque ligne : vérifie stock > quantité demandée
 - Si stock insuffisant → throws BadRequestException (transaction rollback)
 - Crée la Vente, décrémente les stocks, génère le QR code, crée le PDF
 - Transaction commitée
8. VentesController retourne l'objet vente créé
9. ResponseInterceptor intercepte la réponse et l'enveloppe :


```
{ success: true, data: { id, montantTotal, ... }, message: "Vente créée" }
```
10. Réponse HTTP 201 envoyée au client

Dependency Injection (DI) dans NestJS

La **Dependency Injection** est un patron de conception où les dépendances d'une classe sont fournies par le framework plutôt que créées manuellement.

Comment ça fonctionne dans NestJS :

1. Un service est marqué **@Injectable()** → NestJS sait qu'il peut l'injecter
2. Un module déclare ses **providers**: **[MonService]** → NestJS crée une instance singleton
3. Un service ou controller déclare la dépendance dans son constructeur :

```
constructor(private readonly prisma: PrismaService) {}
```

4. NestJS résout automatiquement la dépendance et injecte l'instance

Avantages ici :

- **PrismaService** est créé une seule fois et partagé entre tous les modules
- Les tests unitaires peuvent injecter des mocks à la place des vraies dépendances
- Le couplage entre composants est minimal

3. Structure des dossiers — fichier par fichier

```

pharmacie-backend/
├── .env                                # Variables d'environnement (ne pas committer)
├── .env.example                       # Template des variables (committer)
├── prisma/
│   ├── schema.prisma                 # Schéma de base de données
│   └── seed.ts                       # Script de seed : crée le premier ADMIN (npx prisma db
seed)
├── src/
│   ├── main.ts                       # Point d'entrée de l'application
│   ├── app.module.ts                 # Module racine
│   ├── prisma/
│   │   ├── prisma.module.ts         # Module Prisma (global)
│   │   └── prisma.service.ts        # Service Prisma (connexion DB)
│   ├── common/
│   │   ├── filters/
│   │   │   └── http-exception.filter.ts    # Formateur d'erreurs global
│   │   ├── interceptors/
│   │   │   └── response.interceptor.ts     # Formateur de réponses global
│   │   └── pipes/
│   │       └── validation.pipe.ts         # Validateur global des DTO
│   ├── auth/
│   │   ├── decorators/
│   │   │   ├── public.decorator.ts       # Marqueur de route publique
│   │   │   └── roles.decorator.ts        # Déclarateur de rôles requis
│   │   ├── guards/
│   │   │   ├── jwt-auth.guard.ts         # Vérification du token JWT
│   │   │   └── roles.guard.ts            # Vérification du rôle
│   │   ├── strategies/
│   │   │   ├── jwt.strategy.ts           # Stratégie JWT access token
│   │   │   └── jwt-refresh.strategy.ts    # Stratégie JWT refresh token
│   │   ├── dto/
│   │   │   ├── register.dto.ts          # Validation inscription
│   │   │   └── login.dto.ts             # Validation connexion
│   │   ├── auth.service.ts              # Logique d'authentification
│   │   ├── auth.controller.ts           # Routes d'authentification
│   │   └── auth.module.ts               # Module Auth
│   ├── users/
│   │   ├── dto/
│   │   │   ├── create-user.dto.ts       # Validation création utilisateur
│   │   │   └── update-user.dto.ts        # Validation mise à jour
│   │   ├── users.service.ts             # Logique métier utilisateurs
│   │   ├── users.controller.ts          # Routes utilisateurs
│   │   └── users.module.ts              # Module Users
│   ├── patients/
│   │   ├── dto/
│   │   │   ├── create-patient.dto.ts    # Validation création patient
│   │   │   └── update-patient.dto.ts     # Validation mise à jour

```

```

├── patients.service.ts
├── patients.controller.ts
├── patients.module.ts
├── medicaments/
│   ├── dto/
│   │   ├── create-medicament.dto.ts
│   │   └── update-medicament.dto.ts
│   ├── medicaments.service.ts
│   ├── medicaments.controller.ts
│   └── medicaments.module.ts
├── stock/
│   ├── dto/
│   │   └── update-stock.dto.ts
│   ├── stock.service.ts
│   ├── stock.controller.ts
│   └── stock.module.ts
├── ordonnances/
│   ├── dto/
│   │   ├── create-ordonnance.dto.ts
│   │   └── update-ordonnance.dto.ts
│   ├── ordonnances.service.ts
│   ├── ordonnances.controller.ts
│   └── ordonnances.module.ts
├── ventes/
│   ├── dto/
│   │   └── create-vente.dto.ts
│   ├── ventes.service.ts
│   ├── ventes.controller.ts
│   └── ventes.module.ts
├── qrcode/
│   ├── qrcode.service.ts
│   └── qrcode.module.ts
├── mail/
│   ├── mail.service.ts           # Envoi d'emails (bienvenue, alerte stock, ticket)
│   └── mail.module.ts           # Configuration SMTP Gmail
├── commandes/
│   ├── dto/
│   │   └── create-commande.dto.ts # Validation : lignes[] (medicamentId + quantite) +
note?
│   ├── commandes.service.ts      # Logique : créer, lister, valider, refuser
│   ├── commandes.controller.ts   # Routes : POST, GET, GET/mes-commandes, valider,
refuser
│   └── commandes.module.ts       # Module Commandes
├── notifications/
│   ├── notifications.gateway.ts  # WebSocket Gateway – émet l'événement stock-alerte
│   └── notifications.service.ts  # Service injectable par StockModule et
VentesModule
│   ├── notifications.controller.ts # GET /notifications/events – doc WebSocket dans
Swagger
│   └── notifications.module.ts     # Module exportant NotificationsService

```

Détail de chaque fichier

src/main.ts

- **Rôle** : Point d'entrée de l'application NestJS. Configure globalement les middlewares, pipes, filtres et la doc Swagger.

- **Exporte** : Rien (fichier bootstrap)
- **Importé par** : Node.js directement au démarrage (`node dist/main.js`)
- **Si supprimé** : L'application ne démarre plus

`src/app.module.ts`

- **Rôle** : Module racine qui importe tous les autres modules. C'est la "table des matières" de l'application.
- **Exporte** : `AppModule`
- **Importé par** : `main.ts` via `NestFactory.create(AppModule)`
- **Si supprimé** : L'application ne peut pas être bootstrappée

`src/prisma/prisma.service.ts`

- **Rôle** : Étend `PrismaClient`, gère la connexion/déconnexion à PostgreSQL. Singleton partagé dans toute l'application.
- **Exporte** : `PrismaService`
- **Importé par** : Tous les services métier (`AuthService`, `UserService`, etc.)
- **Si supprimé** : Plus aucun accès à la base de données

`src/prisma/prisma.module.ts`

- **Rôle** : Déclare `PrismaService` comme module global (`@Global()`). Tous les modules peuvent injecter `PrismaService` sans l'importer explicitement.
- **Exporte** : `PrismaModule` + `PrismaService`
- **Importé par** : `AppModule`
- **Si supprimé** : Chaque module devrait importer `PrismaModule` manuellement

`src/common/filters/http-exception.filter.ts`

- **Rôle** : Intercepte toutes les `HttpException` non gérées et les formate en JSON uniforme avec `success: false`.
- **Exporte** : `HttpExceptionFilter`
- **Importé par** : `main.ts` (enregistré globalement)
- **Si supprimé** : Les erreurs retournent le format par défaut de NestJS (incohérent)

`src/common/interceptors/response.interceptor.ts`

- **Rôle** : Intercepte toutes les réponses réussies et les enveloppe dans `{ success: true, data, message }`.
- **Exporte** : `ResponseInterceptor`
- **Importé par** : `main.ts` (enregistré globalement)
- **Si supprimé** : Les réponses retournent les données brutes sans enveloppe

`src/common/pipes/validation.pipe.ts`

- **Rôle** : Configure le `ValidationPipe` global qui valide chaque body entrant contre son DTO.
- **Exporte** : Configuration du pipe
- **Importé par** : `main.ts`
- **Si supprimé** : Les validations `class-validator` ne s'exécutent plus

`src/auth/decorators/public.decorator.ts`

- **Rôle** : Décorateur `@Public()` qui marque une route comme accessible sans token JWT.
- **Exporte** : `Public` decorator et constante `IS_PUBLIC_KEY`
- **Importé par** : `JwtAuthGuard` (pour vérifier si la route est publique) et les controllers

- **Si supprimé** : Impossible de marquer des routes publiques ; login/register deviennent inaccessibles

src/auth/decorators/roles.decorator.ts

- **Rôle** : Décorateur `@Roles(...roles)` qui attache les rôles autorisés aux métadonnées d'une route.
- **Exporte** : `Roles` decorator et constante `ROLES_KEY`
- **Importé par** : `RolesGuard` et les controllers
- **Si supprimé** : Le contrôle d'accès par rôle ne fonctionne plus

src/auth/guards/jwt-auth.guard.ts

- **Rôle** : Guard global qui vérifie le token JWT dans le cookie à chaque requête (sauf routes `@Public()`).
- **Exporte** : `JwtAuthGuard`
- **Importé par** : `AppModule` (enregistré globalement via `APP_GUARD`)
- **Si supprimé** : Toutes les routes deviennent accessibles sans authentification

src/auth/guards/roles.guard.ts

- **Rôle** : Guard qui vérifie que le rôle de l'utilisateur connecté correspond aux rôles requis par `@Roles()`.
- **Exporte** : `RolesGuard`
- **Importé par** : `AppModule` (enregistré globalement via `APP_GUARD`)
- **Si supprimé** : N'importe quel utilisateur authentifié peut accéder à n'importe quelle route

src/auth/strategies/jwt.strategy.ts

- **Rôle** : Stratégie Passport qui extrait le JWT du cookie, le vérifie avec la clé secrète, et place le payload dans `req.user`.
- **Exporte** : `JwtStrategy`
- **Importé par** : `AuthModule`
- **Si supprimé** : `JwtAuthGuard` ne sait plus comment valider un token

src/auth/strategies/jwt-refresh.strategy.ts

- **Rôle** : Stratégie dédiée au refresh token (cookie `refresh_token`). Vérifie avec `JWT_REFRESH_SECRET`.
- **Exporte** : `JwtRefreshStrategy`
- **Importé par** : `AuthModule`
- **Si supprimé** : Le renouvellement silencieux des tokens ne fonctionne plus

4. Concepts NestJS — explications détaillées

Module (`@Module`)

Un module est la **brique de base** de NestJS. Il regroupe des controllers, services et providers liés à une même fonctionnalité.

```
@Module({
  imports: [PrismaModule],    // Modules dont on a besoin
  controllers: [VentesController], // Routes HTTP
  providers: [VentesService],  // Services injectables
  exports: [VentesService],    // Ce qu'on expose aux autres modules
})
export class VentesModule {}
```


Dans ce projet : chaque domaine métier (patients, médicaments, ventes...) a son propre module.

Controller (@Controller, @Get, @Post...)

Le controller est la **porte d'entrée** HTTP. Il reçoit les requêtes, extrait les données (body, params, query), appelle le service, et retourne la réponse.

```
@Controller('ventes')
export class VentesController {
  @Post()
  @Roles('CAISSIER', 'ADMIN')
  create(@Body() dto: CreateVenteDto, @Req() req: Request) {
    return this.ventesService.create(dto, req.user.sub);
  }
}
```

Le controller **ne contient aucune logique métier** — seulement du routage.

Service (@Injectable)

Le service contient toute la **logique métier**. Il est injectable via DI.

```
@Injectable()
export class VentesService {
  constructor(private readonly prisma: PrismaService) {}

  async create(dto: CreateVenteDto, userId: string) {
    // Logique complexe : vérifier stocks, créer vente, décrémenter stocks
  }
}
```

Guard (@UseGuards)

Un guard décide si une requête peut **continuer ou être bloquée** avant d'atteindre le controller.

- **JwtAuthGuard** : "Est-ce que tu as un token JWT valide ?"
- **RolesGuard** : "Est-ce que ton rôle te permet d'accéder à cette route ?"

Les guards s'exécutent **après les middlewares** mais **avant les pipes et controllers**.

Décorateurs personnalisés

@Public() et @Roles() sont des **métadonnées** attachées aux routes via **Reflector**.

```
// Définition
export const Public = () => SetMetadata('isPublic', true);

// Utilisation
@Public()
@Post('login')
login(@Body() dto: LoginDto) { ... }

// Lecture dans le guard
const isPublic = this.reflector.getAllAndOverride<boolean>('isPublic', [
```

```
context.getHandler(),
context.getClass(),
]);
```

Pipe ([ValidationPipe](#))

Un pipe **transforme et valide** les données entrantes avant qu'elles n'atteignent le controller.

Le [ValidationPipe](#) global :

1. Reçoit le body brut { email: "test@test.com", motDePasse: "123" }
2. Instancie le DTO correspondant [LoginDto](#)
3. Exécute les validations [@IsEmail\(\)](#), [@IsString\(\)](#) etc.
4. Si invalide : lance [BadRequestException](#) avec le détail des erreurs
5. Si valide : passe l'objet DTO validé au controller

Interceptor ([ResponseInterceptor](#))

Un interceptor **enveloppe** la requête/réponse. Il peut modifier la réponse avant qu'elle soit envoyée.

```
@Injectable()
export class ResponseInterceptor implements NestInterceptor {
  intercept(context: ExecutionContext, next: CallHandler): Observable<unknown> {
    return next.handle().pipe(
      map((data) => ({
        success: true,
        data: data?.data ?? data,
        message: data?.message ?? 'Opération réussie',
      })),
    );
  }
}
```

Filter ([HttpExceptionFilter](#))

Un filtre **capture les exceptions** non gérées et les transforme en réponses HTTP.

```
@Catch(HttpException)
export class HttpExceptionFilter implements ExceptionFilter {
  catch(exception: HttpException, host: ArgumentsHost) {
    const response = host.switchToHttp().getResponse();
    response.status(status).json({
      success: false,
      statusCode: status,
      message: exception.message,
      timestamp: new Date().toISOString(),
      path: request.url,
    });
  }
}
```

DTO (Data Transfer Object)

Un DTO définit la **forme attendue des données** d'une requête. Les décorateurs `class-validator` décrivent les règles.

```
export class CreatePatientDto {
  @IsString()
  @IsNotEmpty()
  nom: string;

  @IsPhoneNumber('CD') // Numéro congolais
  telephone: string;

  @IsOptional()
  @IsString()
  adresse?: string;
}
```

Strategy (Passport JWT Strategy)

Une stratégie définit **comment extraire et valider** les credentials. Ici, la `JwtStrategy` extrait le token du cookie `access_token` et le vérifie.

```
@Injectable()
export class JwtStrategy extends PassportStrategy(Strategy, 'jwt') {
  constructor(config: ConfigService) {
    super({
      jwtFromRequest: ExtractJwt.fromExtractors([
        (request) => request?.cookies?.access_token,
      ]),
      secretOrKey: config.get('JWT_SECRET'),
    });
  }

  async validate(payload: JwtPayload) {
    return { sub: payload.sub, role: payload.role };
  }
}
```

Middleware vs Guard vs Interceptor vs Filter

Composant	Quand s'exécute	Peut bloquer ?	Accès à	Cas d'usage
Middleware	Avant tout	Oui (next())	req, res	cookie-parser, CORS, logging
Guard	Après middleware	Oui (boolean)	ExecutionContext	Auth, RBAC
Pipe	Avant controller	Oui (exception)	Argument (body/param)	Validation, transformation
Interceptor	Avant ET après controller	Non (observe)	req + réponse	Format réponse, cache, logging
Filter	Sur exception	N/A	Exception + req	Format erreurs

5. Sécurité — explication complète

Pourquoi les tokens sont dans des cookies HTTP-only

Les approches alternatives et leurs risques :

Approche	Risque
Token dans <code>localStorage</code>	Accessible par JavaScript → vulnérable aux attaques XSS
Token dans <code>sessionStorage</code>	Même risque XSS
Token dans header <code>Authorization</code>	Doit être stocké quelque part côté client (retombe sur <code>localStorage</code>)
Cookie HTTP-only	Inaccessible depuis JavaScript. Envoyé automatiquement. Protégé contre XSS.

Configuration utilisée :

```
res.cookie('access_token', accessToken, {
  httpOnly: true,    // JavaScript ne peut pas lire ce cookie
  secure: true,      // HTTPS uniquement (pas envoyé sur HTTP)
  sameSite: 'strict', // Pas envoyé dans les requêtes cross-site (protection CSRF)
  maxAge: 15 * 60 * 1000, // Expire dans 15 minutes
});
```

Différence access token / refresh token

	Access Token	Refresh Token
Durée de vie	15 minutes	7 jours
Cookie	<code>access_token</code>	<code>refresh_token</code>
Stocké en DB ?	Non	Oui (haché dans <code>User.refreshToken</code>)
Usage	Prouver l'identité sur chaque requête	Obtenir un nouvel access token
Compromis	Impact limité (expire vite)	Révocable en BDD

Cycle complet d'authentification

1. LOGIN
POST /auth/login { email, motDePasse }
→ Vérifie le mot de passe avec `bcrypt.compare()`
→ Génère `access_token` (15min) + `refresh_token` (7j)
→ Hache le `refresh_token`, le stocke dans `User.refreshToken`
→ Pose les deux cookies HTTP-only
→ Retourne { success: true, data: { user } }
2. REQUÊTE NORMALE (access token valide)
GET /patients (avec cookie `access_token`)
→ `JwtStrategy` lit le cookie, vérifie la signature
→ `req.user` = { sub: "uuid", role: "CAISSIER" }
→ Controller exécuté normalement
3. ACCESS TOKEN EXPIRÉ
GET /patients → 401 Unauthorized

4. REFRESH

POST /auth/refresh (avec cookie refresh_token)
 → JwtRefreshStrategy valide le refresh_token
 → Vérifie en BDD que le refresh_token haché correspond
 → Génère de nouveaux access_token + refresh_token
 → Rotation : l'ancien refresh_token est invalidé (nouveau haché en BDD)
 → Nouveaux cookies posés

5. LOGOUT

POST /auth/logout
 → User.refreshToken = null en BDD
 → Les deux cookies sont effacés (maxAge: 0)

Hachage bcrypt

```
// À la création du compte
const salt = await bcrypt.genSalt(12); // 12 rounds = ~300ms de calcul
const hash = await bcrypt.hash(motDePasse, salt);
// Stocké en DB : "$2b$12$xyz...abc" (hash irréversible)

// À la connexion
const isValid = await bcrypt.compare(motDePasseFourni, hash);
// bcrypt recalcule le hash et compare (timing-safe)
```

Le refresh token est également haché avant stockage : même si la DB est compromise, l'attaquant ne peut pas utiliser les tokens volés.

Système de rôles (RBAC)

```
ADMIN      → Toutes les routes
PHARMACIEN → Médicaments, Stock, Ordonnances (validation), Ventes (lecture)
PREPARATEUR → Ordonnances (lecture), Stock (lecture/update)
CAISSIER   → Patients, Ventes (création), lecture générale
```

La hiérarchie est implémentée via `@Roles()` sur chaque route et `RolesGuard` qui vérifie.

Routes publiques

Route	Raison
POST /auth/login	L'utilisateur n'est pas encore authentifié
POST /auth/refresh	Utilisé quand l'access token est expiré

6. Prisma — explication complète

Ce qu'est Prisma

Prisma est un **ORM (Object-Relational Mapper) de nouvelle génération** pour Node.js et TypeScript. À partir d'un schéma déclaratif (`schema.prisma`), il génère :

1. Un **client TypeScript typé** (auto-complétion, vérification de types)
2. Des **migrations SQL** (historique des changements de schéma)
3. Un **Prisma Studio** (interface graphique pour explorer les données)

Différence avec les ORMs classiques (TypeORM, Sequelize) :

- TypeORM : décorateurs sur les entités TypeScript → difficile à maintenir, types moins précis
- Prisma : schéma centralisé séparé du code → plus lisible, types 100% précis, meilleure DX

Comment PrismaService est partagé

```
// prisma.module.ts
@Global()                                // Rend le module disponible partout
@Module({
  providers: [PrismaService],
  exports: [PrismaService],              // Expose PrismaService aux autres modules
})
export class PrismaModule {}

// N'importe quel service peut l'injecter :
constructor(private readonly prisma: PrismaService) {}
// Sans avoir besoin d'importer PrismaModule (grâce à @Global())
```

Transaction séquentielle `$transaction([...])`

Définition : Exécute une liste d'opérations Prisma en parallèle dans une seule transaction. Si l'une échoue, toutes sont annulées.

Syntaxe :

```
const [medicament, stock] = await this.prisma.$transaction([
  this.prisma.medicament.create({ data: { nom, prixCDF, ... } }),
  this.prisma.stock.create({ data: { medicamentId: '...', quantite: 0 } }),
]);
```

Cas d'usage dans ce projet : Créer un médicament ET créer son stock initial simultanément. Les deux doivent réussir ou aucun ne doit être créé (cohérence).

Limitation : Les opérations ne peuvent pas dépendre les unes des autres (l'ID du médicament créé n'est pas disponible pour créer le stock dans la même liste). On contourne avec un `uuid` généré à l'avance.

Transaction interactive `$transaction(async tx => {})`

Définition : Exécute une fonction asynchrone dans une transaction. Toutes les opérations utilisant `tx` sont atomiques. Plus flexible : on peut lire, décider, écrire dans la même transaction.

Syntaxe :

```
await this.prisma.$transaction(async (tx) => {
  // 1. Lire le stock actuel
  const stock = await tx.stock.findUnique({ where: { medicamentId } });

  // 2. Vérifier (décision basée sur la lecture)
```

```
if (stock.quantite < quantiteDemandee) {
  throw new BadRequestException('Stock insuffisant');
  // → rollback automatique
}

// 3. Décrémenter le stock
await tx.stock.update({
  where: { medicamentId },
  data: { quantite: { decrement: quantiteDemandee } },
});

// 4. Créer la ligne de vente
await tx.ligneVente.create({ data: { venteId, medicamentId, quantite } });
});
```

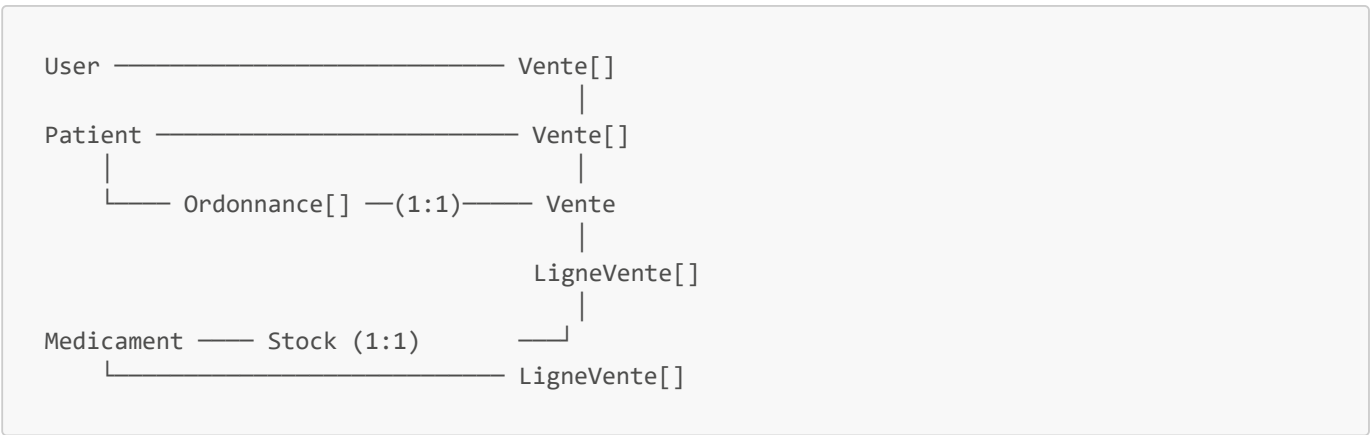
Cas d'usage dans ce projet : Créer une vente avec ses lignes. Il faut lire les stocks, vérifier, puis décrémenter — opérations interdépendantes.

Règle absolue : À l'intérieur d'une transaction interactive, utiliser `tx.xxx` jamais `this.prisma.xxx`. Utiliser `this.prisma` ignorerait la transaction.

Choisir entre les deux types de transactions

Critère	<code>\$transaction([...])</code> séquentielle	<code>\$transaction(async tx => {})</code> interactive
Opérations dépendantes entre elles ?	Non	Oui
Besoin de lire avant d'écrire ?	Non	Oui
Logique conditionnelle	Non	Oui
Performance	Légèrement meilleure	Standard
Ce projet	Créer médicament + stock	Créer vente + décrémenter stocks

Relations entre modèles



Lecture : Un **Patient** peut avoir plusieurs **Ordonnances** et plusieurs **Ventes**. Une **Vente** appartient à un **User** (caissier) et peut être liée à un **Patient** et une **Ordonnance**. Une **Vente** a plusieurs **LigneVentes**, chacune liée à un **Medicament**. Chaque **Medicament** a exactement un **Stock**.

AuthModule

Responsabilité : Authentification des utilisateurs, gestion des tokens JWT, cookies, changement de mot de passe.

Méthode	Route	Rôles	Description
POST	/auth/register	@Public	Créer un compte client — rôle CLIENT attribué automatiquement
POST	/auth/login	@Public	Se connecter, recevoir les cookies
POST	/auth/refresh	@Public	Renouveler l'access token
POST	/auth/logout	Authentifié	Se déconnecter, effacer les cookies
GET	/auth/me	Authentifié	Profil de l'utilisateur connecté
PATCH	/auth/change-password	Authentifié	Changer son propre mot de passe

DTOs :

- **RegisterDto** : **nom**, **prenom**, **email** (IsEmail), **motDePasse** (MinLength 8) — **pas de champ role**
- **LoginDto** : **email** (IsEmail), **motDePasse** (IsString)
- **ChangePasswordDto** : **ancienMotDePasse** (IsString), **nouveauMotDePasse** (MinLength 8)

Règle métier clé : **register** force **role**: **CLIENT** — impossible de s'auto-attribuer un rôle staff.

Transactions utilisées : Aucune (opérations simples sur **User**)

UsersModule

Responsabilité : Gestion du personnel de la pharmacie (CRUD). Import de **MailModule** pour l'envoi des identifiants.

Méthode	Route	Rôles	Description
POST	/users	ADMIN	Créer un compte staff — mot de passe généré automatiquement et envoyé par email
GET	/users	ADMIN	Liste tous les utilisateurs
GET	/users/:id	ADMIN	Détail d'un utilisateur
PATCH	/users/:id	ADMIN	Modifier nom/prénom/email/rôle (pas le mot de passe)
DELETE	/users/:id	ADMIN	Supprimer un utilisateur

DTOs :

- **CreateUserDto** : **nom**, **prenom**, **email**, **role** (ADMIN/PHARMACIEN/PREPARATEUR/CAISSIER — **CLIENT** exclu) — **pas de motDePasse**
- **UpdateUserDto** : **nom?**, **prenom?**, **email?**, **role?** — le mot de passe n'est pas modifiable ici

Logique de création :

1. L'admin fournit nom, prénom, email, rôle
2. **UsersService** génère un mot de passe temporaire sécurisé (12 chars, maj + chiffre + spécial)
3. Le mot de passe est haché (bcrypt 12 rounds) et stocké
4. **MailService.sendCredentials()** envoie email + mot de passe en clair à l'employé
5. L'employé doit changer son mot de passe via **PATCH /auth/change-password**

PatientsModule

Responsabilité : Gestion des patients (fiches clients).

Méthode	Route	Rôles	Description
POST	/patients	CAISSIER, ADMIN, PHARMACIEN	Créer un patient
GET	/patients	Tous rôles	Lister les patients
GET	/patients/:id	Tous rôles	Détail d'un patient
PATCH	/patients/:id	CAISSIER, ADMIN	Modifier un patient
DELETE	/patients/:id	ADMIN	Supprimer un patient

DTOs :

- `CreatePatientDto` : `nom`, `prenom`, `telephone` (unique, format congolais), `adresse`?

MedicamentsModule

Responsabilité : Catalogue des médicaments avec prix en CDF et USD. Les routes GET sont **publiques** (aucun token requis) pour permettre aux clients de parcourir le catalogue avant de s'inscrire ou de passer commande.

Méthode	Route	Rôles	Description
POST	/medicaments	ADMIN, PHARMACIEN	Créer un médicament + son stock initial
GET	/medicaments	@Public	Lister les médicaments — catalogue accessible sans authentification
GET	/medicaments/:id	@Public	Détail d'un médicament avec stock — accessible sans authentification
PATCH	/medicaments/:id	ADMIN, PHARMACIEN	Modifier un médicament
DELETE	/medicaments/:id	ADMIN	Supprimer un médicament

Transaction séquentielle : Lors de la création, `Medicament` et son `Stock` initial sont créés dans la même transaction séquentielle.

StockModule

Responsabilité : Consultation, mise à jour manuelle des stocks, et génération/envoi des bons de commande fournisseur. Dépend de `NotificationsModule` (alertes WebSocket) et `MailModule` (envoi email).

Méthode	Route	Rôles	Description
GET	/stock	Tous rôles	Liste tous les stocks
GET	/stock/alertes	Tous rôles	Médicaments sous le seuil minimum
GET	/stock/rapport-commande	ADMIN, PHARMACIEN, PREPARATEUR	Prévisualiser le bon de commande
POST	/stock/rapport-commande/envoyer	ADMIN, PHARMACIEN	Générer (PDF ou Excel) et envoyer au fournisseur

Méthode	Route	Rôles	Description
GET	<code>/stock/:medicamentId</code>	Tous rôles	Stock d'un médicament
PATCH	<code>/stock/:medicamentId</code>	ADMIN, PHARMACIEN, PREPARATEUR	Ajuster le stock (réapprovisionnement)

Workflow bon de commande :

1. GET `/stock/rapport-commande` → le pharmacien voit la liste des produits sous seuil + quantités suggérées
2. Il valide ou ajuste les quantités dans l'interface frontend
3. POST `/stock/rapport-commande/envoyer` → génère le fichier et l'envoie à l'email du fournisseur choisi

Logique de suggestion : $\text{quantiteACommander} = \max(\text{seuil} \times 2 - \text{stockActuel}, \text{seuil})$ — on commande assez pour atteindre le double du seuil minimum.

Génération PDF : `pdfkit` — document A4, tableau coloré, saut de page automatique.

Génération Excel : `exceljs` — feuille formatée, lignes alternées, cellule jaune si `quantiteActuelle = 0`.

Contrainte : le fournisseur doit avoir un email enregistré (champ `email` dans `Fournisseur`). Si absent → `400 Bad Request` avec message explicatif.

OrdonnancesModule

Responsabilité : Gestion du cycle de vie des ordonnances médicales.

Méthode	Route	Rôles	Description
POST	<code>/ordonnances</code>	CAISSIER, PHARMACIEN, ADMIN	Enregistrer une ordonnance
GET	<code>/ordonnances</code>	Tous rôles	Lister les ordonnances
GET	<code>/ordonnances/:id</code>	Tous rôles	Détail d'une ordonnance
PATCH	<code>/ordonnances/:id/valider</code>	PHARMACIEN, ADMIN	Valider une ordonnance
PATCH	<code>/ordonnances/:id/refuser</code>	PHARMACIEN, ADMIN	Refuser une ordonnance

VentesModule

Responsabilité : Processus de vente complet avec décrétement de stock et génération de ticket.

Méthode	Route	Rôles	Description
POST	<code>/ventes</code>	CAISSIER, ADMIN	Créer une vente
GET	<code>/ventes</code>	Tous rôles	Lister les ventes
GET	<code>/ventes/:id</code>	Tous rôles	Détail d'une vente
GET	<code>/ventes/:id/ticket</code>	Tous rôles	Télécharger le ticket PDF
PATCH	<code>/ventes/:id/annuler</code>	ADMIN	Annuler une vente

Transaction interactive : La création d'une vente utilise une transaction interactive car elle :

1. Lit les stocks de chaque médicament
2. Vérifie que les quantités sont disponibles
3. Crée la vente et ses lignes

4. Décrémente les stocks

QrCodeModule

Responsabilité : Services de génération QR code (PNG base64) et ticket PDF.

Pas de routes HTTP directes. Le `QrCodeService` est injecté dans `VentesService`.

FournisseursModule

Responsabilité : Gestion du répertoire des fournisseurs de médicaments.

Méthode	Route	Rôles	Description
POST	<code>/fournisseurs</code>	ADMIN, PHARMACIEN	Créer un fournisseur
GET	<code>/fournisseurs</code>	Tous rôles	Lister les fournisseurs
GET	<code>/fournisseurs/:id</code>	Tous rôles	Détail d'un fournisseur
PATCH	<code>/fournisseurs/:id</code>	ADMIN, PHARMACIEN	Modifier un fournisseur
DELETE	<code>/fournisseurs/:id</code>	ADMIN	Supprimer un fournisseur

DTOs :

- `CreateFournisseurDto` : `nom`, `telephone` (unique), `email?` (unique), `adresse?`
- `UpdateFournisseurDto` : Tous les champs optionnels (extends `PartialType`)

Modèle Prisma :

```
Fournisseur { id, nom, telephone (unique), email (unique?), adresse?, createdAt, updatedAt }
```

CommandesModule

Responsabilité : Système de commandes clients. Le client passe une commande en ligne ; le staff (pharmacien ou admin) la valide ou la refuse. Aucune déduction de stock à la commande — le stock n'est affecté que lors d'une vente physique (`POST /ventes`).

Méthode	Route	Rôles	Description
POST	<code>/commandes</code>	CLIENT	Passer une commande (lignes de médicaments + quantités)
GET	<code>/commandes</code>	ADMIN, PHARMACIEN, CAISSIER, PREPARATEUR	Lister toutes les commandes
GET	<code>/commandes/mes-commandes</code>	CLIENT	Voir uniquement ses propres commandes
GET	<code>/commandes/:id</code>	Authentifié	Détail (CLIENT = ses propres seulement, sinon 403)
PATCH	<code>/commandes/:id/valider</code>	ADMIN, PHARMACIEN	Valider une commande <code>EN_ATTENTE</code>
PATCH	<code>/commandes/:id/refuser</code>	ADMIN, PHARMACIEN	Refuser une commande <code>EN_ATTENTE</code>

DTOs :

- `CreateCommandeDto` : `lignes[]` (`medicamentId` UUID + `quantite` ≥ 1), `note?` — via `LigneCommandeDto` imbriqué, validé avec `@ValidateNested` + `@Type`
- Pas de DTO de mise à jour : les commandes ne se modifient pas, elles se valident ou se refusent

Modèles Prisma :

```

Commande      { id, clientId (→ User), statut (EN_ATTENTE|VALIDEE|REFUSEE), note?,
                createdAt, updatedAt }
LigneCommande { id, commandeId (→ Commande, onDelete: Cascade), medicamentId (→
Medicament), quantite }

```

Relations :

```

User (CLIENT) — Commande[] — LigneCommande[] — Medicament

```

Contrôle d'accès sur `GET /commandes/:id` : géré dans `CommandesService.findOne()` — si `requesterRole === CLIENT` et `commande.clientId !== requesterId` → `ForbiddenException`.

Note sur l'ordre des routes : `GET /commandes/mes-commandes` est déclaré **avant** `GET /commandes/:id` dans le controller pour éviter que NestJS interprète `mes-commandes` comme un paramètre `:id`.

NotificationsModule

Responsabilité : Alertes de stock en temps réel via WebSocket (Socket.IO). Pousse l'événement `stock-alerte` à tous les clients connectés dès qu'un stock descend sous son seuil minimum.

Fichier	Rôle
<code>notifications.gateway.ts</code>	<code>@WebSocketGateway</code> — détient le <code>Server Socket.IO</code> et expose <code>emitStockAlerte()</code>
<code>notifications.service.ts</code>	Service injectable — appelé par <code>StockService</code> et <code>VentesService</code>
<code>notifications.controller.ts</code>	<code>GET /notifications/events</code> — retourne la documentation du contrat WebSocket

Déclencheurs de l'événement `stock-alerte` :

Contexte	Condition
<code>VentesService.create()</code>	Stock restant après vente $\leq$ <code>seuilMinimum</code> du médicament
<code>StockService.update()</code>	Nouvelle quantité après mise à jour manuelle $\leq$ <code>seuilMinimum</code>

Payload de l'événement :

```

{
  "medicamentId": "uuid",
  "nom": "Paracétamol 500mg",
  "quantite": 5,
  "seuilMinimum": 10,

```

```

"message": "Stock critique : Paracétamol 500mg – 5 unité(s) restante(s) (seuil : 10)",
"timestamp": "2026-05-22T10:30:00.000Z"
}

```

Note : Le module fonctionne sur le même port que l'API REST. Aucune configuration supplémentaire côté serveur n'est requise.

MailModule

Responsabilité : Envoi d'emails transactionnels via Gmail SMTP. Aucune route HTTP exposée — utilisé comme service utilitaire par les autres modules.

Dépendances : @nestjs-modules/mailer, nodemailer, ConfigModule

Méthodes disponibles dans MailService :

Méthode	Appelé automatiquement	Déclencheur	Description
sendCredentiels(email, prenom, role, mdpTemp)	✅ Oui	UserService.create()	Envoie email + mot de passe temporaire au nouvel employé
sendMail({ to, subject, html })	❌ Non	Générique	Email HTML libre
sendBienvenue(email, nom)	❌ Non	À brancher si besoin	Email de bienvenue générique
sendAlerteStock(email, médicaments[])	❌ Non	À brancher dans StockService	Alerte email quand stock ≤ seuil minimum
sendTicketVente(email, venteId, montant, devise)	❌ Non	À brancher dans VentesService	Reçu de vente par email

Configuration : SMTP Gmail avec mot de passe d'application (pas le mot de passe du compte Google). Pour utiliser MailService dans un autre module, ajouter MailModule aux imports du module concerné. Exemple : UsersModule l'importe pour envoyer les identifiants.

8. QR Code et génération de tickets PDF

Génération du QR Code

La bibliothèque qrcode génère un QR code PNG encodé en base64.

```

// Ce que le QR code encode : simplement l'ID de la vente
const qrData = `PHARMACIE-VENTE-${venteId}`;
const qrBase64 = await QRCode.toDataURL(qrData, {
  errorCorrectionLevel: 'H', // Haute résistance aux dommages
  width: 200,
});
// Résultat : "data:image/png;base64,iVBORw0KGgo..."

```

Utilité du QR code : Le pharmacien peut scanner le QR code pour retrouver instantanément la vente dans le système et vérifier son authenticité (évite les faux tickets).

Génération du PDF avec pdfkit

```
const doc = new PDFDocument({ size: 'A6', margin: 20 });

// En-tête
doc.fontSize(16).text('PHARMACIE DIGITALE RDC', { align: 'center' });
doc.fontSize(10).text(`Date : ${new Date().toLocaleDateString('fr-CD')}`);

// Tableau des médicaments vendus
lignes.forEach(ligne => {
  doc.text(`${ligne.medicament.nom} x${ligne.quantite} = ${ligne.prixTotal} ${ligne.devises}`);
});

// Montant total
doc.fontSize(14).text(`TOTAL : ${montantTotal} ${devises}`);

// QR Code (image base64)
const qrImageBuffer = Buffer.from(qrBase64.split(',')[1], 'base64');
doc.image(qrImageBuffer, { width: 80, align: 'center' });
```

Contenu complet du ticket PDF

1. Nom de la pharmacie
2. Date et heure de la vente
3. Nom du caissier
4. Nom du patient (si enregistré)
5. Liste des médicaments : nom, quantité, prix unitaire, sous-total
6. Montant total + devise
7. QR code (encodant l'ID de la vente)
8. Numéro de vente (ID)

9. Format uniforme des réponses API

Réponse succès

```
{
  "success": true,
  "data": {
    "id": "uuid-de-la-vente",
    "montantTotal": "45000.00",
    "devises": "CDF",
    "statut": "FINALISEE"
  },
  "message": "Vente créée avec succès"
}
```

Réponse erreur

```
{
  "success": false,
  "statusCode": 400,
  "message": "Stock insuffisant pour Paracétamol 500mg (disponible: 3, demandé: 10)",
  "timestamp": "2024-01-15T10:30:00.000Z",
  "path": "/api/ventes"
}
```

Comment le ResponseInterceptor fonctionne

```
// Avant interceptor : le controller retourne
return { data: vente, message: 'Vente créée' };

// Après interceptor :
{
  success: true,
  data: vente,
  message: 'Vente créée'
}
```

Si le controller retourne directement un objet (sans `data/message`), l'interceptor l'encapsule :

```
return vente;
// Devient :
{ success: true, data: vente, message: 'Opération réussie' }
```

10. Variables d'environnement

Variable	Rôle	Type	Exemple
<code>DATABASE_URL</code>	URL de connexion PostgreSQL	string	<code>postgresql://user:pass@localhost:5432/pharmacie_db</code>
<code>JWT_SECRET</code>	Clé de signature des access tokens	string	<code>super-secret-key-256-bits-minimum</code>
<code>JWT_EXPIRES_IN</code>	Durée de vie de l'access token	string	<code>15m</code>
<code>JWT_REFRESH_SECRET</code>	Clé de signature des refresh tokens	string	<code>autre-secret-refresh-key</code>
<code>JWT_REFRESH_EXPIRES_IN</code>	Durée de vie du refresh token	string	<code>7d</code>

Variable	Rôle	Type	Exemple
PORT	Port d'écoute du serveur	number	3000
NODE_ENV	Environnement d'exécution	string	development ou production
PHARMACIE_NOM	Nom affiché sur les tickets PDF	string	Pharmacie Centrale Kinshasa
EMAIL_USER	Adresse Gmail expéditrice	string	pharmacie@gmail.com
EMAIL_APP_PASSWORD	Mot de passe d'application Gmail (16 chars)	string	ablq wtaq kdnq kuqw
TUNNEL_HOSTNAME	Sous-domaine loophole pour affichage au démarrage	string	pharmacie
ADMIN_EMAIL	Email du premier administrateur (seed uniquement)	string	admin@pharmacie.cd
ADMIN_MOT_DE_PASSE	Mot de passe du premier administrateur (seed uniquement)	string	MotDePasse@2025
ADMIN_NOM	Nom de famille de l'admin seed	string	Admin (défaut)
ADMIN_PRENOM	Prénom de l'admin seed	string	Principal (défaut)

i Les variables `ADMIN_*` ne sont utilisées **que** par `npx prisma db seed` — le serveur NestJS ne les lit jamais. Elles ne font pas partie du schéma de validation Joi dans `app.module.ts`.

Chargement et validation avec Joi

```
// app.module.ts
ConfigModule.forRoot({
  validationSchema: Joi.object({
    DATABASE_URL: Joi.string().required(),
    JWT_SECRET: Joi.string().min(32).required(),
    JWT_EXPIRES_IN: Joi.string().default('15m'),
    JWT_REFRESH_SECRET: Joi.string().min(32).required(),
    JWT_REFRESH_EXPIRES_IN: Joi.string().default('7d'),
```



```
PORT: Joi.number().default(3000),
NODE_ENV: Joi.string().valid('development', 'production',
'test').default('development'),
}),
})
```

Si une variable requise manque, l'application refuse de démarrer avec un message d'erreur clair.

Accès dans un service

```
constructor(private readonly config: ConfigService) {}

const secret = this.config.get<string>('JWT_SECRET');
const port = this.config.get<number>('PORT');
```

11. Guide de démarrage rapide

Prérequis

- Node.js >= 20
- npm >= 9
- PostgreSQL >= 14
- (optionnel) pgAdmin pour visualiser la DB

Étapes

```
# 1. Copier le fichier d'environnement
cp .env.example .env

# 2. Éditer .env avec vos valeurs
# DATABASE_URL="postgres://postgres:monmotdepasse@localhost:5432/pharmacie_rdc"
# JWT_SECRET="votre-cle-secrete-tres-longue-32-chars-minimum"
# JWT_REFRESH_SECRET="autre-cle-refresh-32-chars-minimum"
# ADMIN_EMAIL="admin@pharmacie.cd"
# ADMIN_MOT_DE_PASSE="MotDePasseAdmin@2025"

# 3. Créer la base de données dans PostgreSQL
# (via psql ou pgAdmin : CREATE DATABASE pharmacie_rdc;)

# 4. Appliquer le schéma Prisma à la base de données
npx prisma migrate dev --name init

# 5. Créer le premier administrateur (opération unique, idempotente)
npx prisma db seed

# 6. (Optionnel) Voir les données dans Prisma Studio
npx prisma studio

# 7. Démarrer le serveur en mode développement
npm run start:dev
```

À propos du seed : le script `prisma/seed.ts` vérifie si un ADMIN existe déjà avant d'en créer un. Il est donc sûr à relancer plusieurs fois — il ne créera jamais de doublon.

Documentation Swagger

Une fois le serveur démarré :

- URL : `http://localhost:3000/api/docs`
- Interface interactive pour tester toutes les routes

Tester l'authentification avec Swagger

1. Aller sur `http://localhost:3000/api/docs`
2. Le premier admin a été créé par `npx prisma db seed` avec l'email défini dans `.env`
3. Trouver `POST /auth/login` → se connecter avec les identifiants admin
4. Le cookie `access_token` est posé automatiquement par le navigateur
5. Toutes les routes protégées fonctionnent maintenant dans Swagger
6. Pour créer des comptes staff : `POST /users` (réservé ADMIN)

12. Glossaire

Terme	Définition
JWT	JSON Web Token. Token signé contenant un payload JSON. Se divise en header.payload.signature. Non révocable, donc durée de vie courte.
Cookie HTTP-only	Cookie que JavaScript ne peut pas lire (<code>document.cookie</code> ne le voit pas). Envoyé automatiquement par le navigateur avec chaque requête.
RBAC	Role-Based Access Control. Contrôle d'accès basé sur les rôles. Chaque utilisateur a un rôle, chaque route exige un ou plusieurs rôles.
DTO	Data Transfer Object. Objet qui définit la structure attendue des données d'une requête. Sert aussi à la validation et à la documentation Swagger.
Guard	Composant NestJS qui décide si une requête peut continuer. Retourne <code>true</code> (passe) ou <code>false</code> /exception (bloqué).
Interceptor	Composant NestJS qui enveloppe l'exécution d'une route. Peut modifier la requête avant et la réponse après.
Filter	Composant NestJS qui capture les exceptions et les transforme en réponses HTTP formatées.
Pipe	Composant NestJS qui transforme et valide les données entrantes (body, params, query).
Strategy	Terme Passport.js. Définit comment extraire et vérifier des credentials (JWT depuis cookie, username/password, OAuth...).
Transaction	Ensemble d'opérations DB exécutées atomiquement : soit toutes réussissent, soit toutes sont annulées (rollback).
Prisma	ORM TypeScript générant un client typé depuis un schéma déclaratif. Remplace TypeORM ou Sequelize.
ORM	Object-Relational Mapper. Traduit des opérations objet TypeScript en requêtes SQL.
CRUD	Create, Read, Update, Delete. Les quatre opérations de base sur les données.

Terme	Définition
REST	Representational State Transfer. Architecture d'API où chaque ressource a une URL, et les opérations CRUD correspondent aux verbes HTTP (GET, POST, PATCH, DELETE).
Swagger	Outil de documentation et test d'API REST. NestJS génère la doc automatiquement depuis les décorateurs.
bcrypt	Algorithme de hachage de mots de passe résistant aux attaques GPU. Intègre un sel aléatoire et un facteur de coût ajustable.
Decimal	Type Prisma pour les montants financiers. Évite les erreurs d'arrondi des flottants ($0.1 + 0.2 \neq 0.3$ en float).
Refresh Token Rotation	À chaque refresh, un nouveau refresh token est généré et l'ancien invalidé. Limite l'impact d'un token volé.
Singleton	Pattern où une seule instance d'une classe existe dans toute l'application. PrismaService est un singleton.
DI (Dependency Injection)	Pattern où une classe reçoit ses dépendances de l'extérieur plutôt que de les créer. NestJS gère automatiquement les instances.
Migration	Fichier SQL généré par Prisma représentant une modification du schéma de DB. Permet de versionner l'évolution de la base.
WebSocket	Protocole de communication bidirectionnel persistant sur une connexion TCP. Contrairement à HTTP, le serveur peut pousser des données au client sans requête préalable.
Socket.IO	Bibliothèque WebSocket avec reconnexion automatique, rooms, namespaces et fallback polling. Utilisée ici via <code>@nestjs/websockets</code> et <code>@nestjs/platform-socket.io</code> .
Gateway (NestJS)	Composant NestJS qui gère les connexions WebSocket. Équivalent d'un Controller pour les messages temps réel. Annoté <code>@WebSocketGateway()</code> .
seuilMinimum	Quantité seuil définie par médicament dans <code>Stock</code> . Quand <code>quantite ≤ seuilMinimum</code> , une alerte WebSocket <code>stock-alerte</code> est émise automatiquement.
Mot de passe temporaire	Mot de passe de 12 caractères généré par <code>crypto.randomBytes</code> lors de la création d'un compte staff. Envoyé en clair par email une seule fois. Le staff doit le changer via <code>PATCH /auth/change-password</code> .
Fournisseur	Entité représentant un distributeur ou grossiste en médicaments. Stocké dans la table <code>Fournisseur</code> (indépendante des médicaments pour l'instant).
Commande	Demande de médicaments soumise par un client via <code>POST /commandes</code> . Différent d'une <code>Vente</code> : la commande n'affecte pas le stock, elle représente une intention d'achat que le staff valide ou refuse.
StatutCommande	Enum des états d'une commande : <code>EN_ATTENTE</code> (créée), <code>VALIDEE</code> (acceptée par le staff), <code>REFUSEE</code> (rejetée).
Seed (prisma db seed)	Script d'initialisation de la base de données (<code>prisma/seed.ts</code>). Crée le premier ADMIN depuis les variables <code>.env</code> . Idempotent : ne crée rien si un ADMIN existe déjà. Commande : <code>npm run prisma db seed</code> .
Route publique (@Public)	Route décorée avec <code>@Public()</code> qui bypass le <code>JwtAuthGuard</code> . Aucun token requis. Utilisé pour : <code>POST /auth/login</code> , <code>POST /auth/register</code> , <code>GET /medicaments</code> , <code>GET /medicaments/:id</code> .

Terme	Définition
Bon de commande	Document (PDF ou Excel) listant les médicaments à réapprovisionner avec les quantités souhaitées. Prévisualisé via GET /stock/rapport-commande , généré et envoyé par email au fournisseur via POST /stock/rapport-commande/envoyer .
exceljs	Bibliothèque Node.js pour créer des fichiers Excel (.xlsx). Utilisée pour le bon de commande fournisseur en format tableur. Les quantités urgentes (stock = 0) sont mises en évidence en jaune.